

11. 2D vektorová grafika: metody rasterizace úseček a polygonů, reprezentace pomocí Bézierovy křivky

11. 2D vektorová grafika

SZZ okruh 11 (FIT BUT). **Vektorová** grafika ukládá data analyticky (úsečky, křivky, polygony) — neomezená přesnost, snadná editace; **rastrová** ukládá pixely. **Rasterizace** = převod vektoru na rastr.

Shrnutí

Rasterizace úsečky

- Tvary přímky: obecný ($Ax+By+C=0$, $[A,B]$ normálový vektor), parametrický, směrnice ($y = kx + q$).
- **DDA** — floating point; přírůstek $dx = 1$, $dy =$ směrnice k , Y se zaokrouhluje ($+0,5$). Existuje i fixed-point varianta.
- **Bresenham (midpoint)** — celočíselná aritmetika, o posunu v Y rozhoduje znaménko **prediktoru**; snadná HW realizace **nejpoužívanější**.
- **Kružnice** — počítá se $\frac{1}{8}$ a dopočítá symetrii; midpoint algoritmus (Bresenham pro kružnici).

[[media/szz-11/media/image25.png]] *Bresenhamův algoritmus — odvození rozhodovacího prediktoru P_i (test znaménka).*

Vyplňování polygonů

- **Scanline fill** — průchod po řádcích, lichý počet průsečíků = obarvovat; problémy s lokálními extrémy a vodorovnými hranami.
- **Inverzní řádkové** — invertování pixelů napravo od průsečíku každé hrany.
- **Pinedův algoritmus** — jen konvexní (trojúhelníky); pixel se obarví, leží-li v kladné polorovině všech hran; snadná HW realizace.
- **Semínkové vyplňování** — šíření z bodu na sousedy (4-okolí / 8-okolí), rekurze nebo fronta.

[[media/szz-11/media/image7.png]] *Pinedův algoritmus — hranová funkce $E_i(x,y)$ jako vektorový součin (kladná/záporná polorovina hrany).*

Křivky

- **Interpolační** (prochází body) $\times$ **aproximační** (Bézier).
- **Bézierova křivka** — aproximační, **Bernsteinovy polynomy**, stupeň n určen $n+1$ body, prochází koncovými body; využití ve fontech.
- **de Casteljau** — rekurzivní vykreslení dělením úseků v poměru $t : (1-t)$.
- **Bézierovy kubiky** — 3. stupeň, segment ze 4 řídicích bodů; navazování vyžaduje totožné koncové body a stejné tečné vektory.

[[media/szz-11/media/image5.png]] *Algoritmus de Casteljau — rekurzivní dělení řídicího polygonu.*

Souvislosti

Navazuje na [[okruhy/12-3d-transformace-pipeline|3D grafiku]] (rasterizace trojúhelníků); matematický základ tvoří [[okruhy/17-diferencialni-integralni-pocet|polynomy a derivace]].

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Rasterizace a úsečka □ [[#Rasterizace úsečky]] - *Co je rasterizace, proč?* □ Převod vektoru na pixely; displeje jsou rastrové. Vektor má neomezenou přesnost a snadné transformace. - *Rovnice úsečky?* □ obecná ($Ax+By+C=0$), parametrická, směrnice ($y=kx+q$); k = tangens úhlu. - *DDA vs. Bresenham?* □ DDA: float, dy = směrnice k (zaokrouhlování); Bresenham/midpoint: celočíselný, o kroku v Y rozhoduje znaménko prediktoru □ rychlejší, vhodný pro HW. - *Oktanty?* □ Algoritmus platí pro jeden oktant, ostatní se převedou symetrií. - *Kružnice?* □ Midpoint algoritmus: počítá $\frac{1}{8}$, zbytek symetrií, celočíselně.

Vyplňování polygonů □ [[#Vyplňování polygonů]] - *Řádkové vyplňování?* □ Po řádcích; lichý počet průsečíků s hranami □ obarvit. - *Pinedův algoritmus?* □ Jen konvexní (trojúhelník); pixel se obarví, leží-li v kladné polorovině všech hran. - *Semínkové (flood fill)?* □ Šíření z bodu na sousedy (4-/8-okolí), rekurze nebo fronta.

Křivky □ [[#Křivky]] - *Aproximační vs. interpolační?* □ Aproximační (Bézier) neprochází řídicími body; interpolační jimi prochází. - *Bézier a de Casteljau?* □ Bézier = Bernsteinovy polynomy, prochází koncovými body; de Casteljau = rekurzivní dělení v poměru $t:(1-t)$.

Plné znění (ke studiu)

Vektorová grafika

Zpracovávané a zobrazované informace popisujeme a **ukládáme analyticky** (spojitě) ve formě skupiny **vektorových entit** (úsečky, kružnice, křivky, polygony, atd.)

- Přesnost popisu je teoreticky neomezená.
- Vlastnosti uložených objektů (obrázků) lze kdykoliv jednoduše měnit.
- SVG

Rastrová grafika

Zpracovávané a zobrazované informace popisujeme a **ukládáme diskrétně** ve formě **rastrové matice** (2D/3D) po **pixelech**.

- Nelze jednoduše měnit vlastnosti uložených objektů.
- Daná neměnná přesnost (rozlišení), nelze jednoduše měnit.
- PNG, JPG

Rasterizace

Proces převodu vektorové reprezentace dat na rastrovou formu.

Úsečka

Základní vektorová entita definovaná 2 body (konce). Všechny zmíněné algoritmy pracují **správně** pouze v **1. kvadrantu** a jen pokud úsečka **roste** rychleji ve směru osy **X** (má max. sklon **45 stupňů**),

ve směru **osy Y** také musí **růst**. (Ostatní úsečky je možné do první poloviny 1. kvadrantu převést, vykreslit a převést zpátky do původní polohy a orientace.)

- **Obecný tvar** - $Ax + By + C = 0$, kde $A = (y_1 - y_2)$, $B = (x_2 - x_1)$, **POZOR** vektor $[A, B]$ je **normálovým vektorem** přímky.
- **Parametrický tvar** - $x = x_1 + tA$, $y = y_1 + tB$, kde $t \in (-\infty, \infty)$ a $A = (x_2 - x_1)$, $B = (y_2 - y_1)$, vektor $[A, B]$ je **směrovým vektorem** přímky.
- **Směrnice tvar** - $y = kx + q$, kde $k = (y_2 - y_1) / (x_2 - x_1)$

Algoritmus DDA (Digital differential analyzer)

Výpočet je ve **floating point** aritmetice. Postup je následující:

- Vykresluje úsečku po pixelech od bodu **P1** k bodu **P2**.
- V **ose X** postupujeme s přírůstkem **dx = 1**.
- V **ose Y** je přírůstek dán **velikostí směrnice** ($k = (y_2 - y_1) / (x_2 - x_1)$) úsečky.
- Souřadnice **Y** se **zaokrouhluje** na **nejbližší celé číslo** - (přičíst **0.5 před** konverzí na **int**).

DDA s fixed-point aritmetikou

[[media/szz-11/media/image9.png]]

Používá bitový posun a eliminuje tak nutnost používat floating point, jinak pracuje na stejném principu jako floating point DDA. [[media/szz-11/media/image6.png]]

Bresenhamův algoritmus (midpoint algoritmus)

Algoritmus používá celočíselnou aritmetiku. Je jednodušší pro HW implementaci. Dnes je tudíž **nejpoužívanější**. Postup:

- Vykresluje úsečku po pixelech od bodu **P1** k bodu **P2**.
- V **ose X** postupujeme s přírůstkem **dx = 1**.
- O posunu v **ose Y** rozhodujeme podle znaménka tzv. **prediktoru**.

[[media/szz-11/media/image25.png]]

[[media/szz-11/media/image18.png]]

[[media/szz-11/media/image23.png]]

Kružnice

[[media/szz-11/media/image26.png]]

[[media/szz-11/media/image2.png]]

Definována souřadnicí **středu** a **poloměrem**:

$$(x - s_1)^2 + (y - s_2)^2 = r^2$$

Výpočty se provádí pro $\frac{1}{8}$ **kružnice**, zbylá část se dopočítá díky **symetrii**. Algoritmy jsou odvozeny pro kružnici se **středem v počátku** $[0, 0]$.

Vykreslení po bodech (plyne z rovnice kružnice)

Nejjednodušší pro pochopení a implementaci, ale náročné pro HW zpracování. Pracuje se s **desetinými** čísly (floating point). Vykresluje se oktant **[x,y]** na obrázky. **cx** je **x-ová** souřadnice **středu** a **cy** je **y-ová** souřadnice **středu**. Algoritmus vykresluje ve směru hodinových ručiček následovně:

- Jdeme po pixelu od bodu [0, R], dokud není **x = y**.
- V **ose X** postupujeme s přírůstkem **dx = 1**.
- Pozici v **ose Y** vypočteme po každé změně **X** podle vztahu $y = \sqrt{R^2 - x^2}$
- Souřadnice **Y** se **zaokrouhluje** na **nejbližší celé** číslo - (přičíst **0.5 před** konverzí na **int**).

[[media/szz-11/media/image8.png]]

[[media/szz-11/media/image24.png]]

Vykreslení jako n-úhelník

Použitím DDA algoritmu se kružnice vykreslí jako by to byl **n-úhelník** (s větším poloměrem je potřeba větší n) pomocí **úseček**. Využívá k tomu matici otočení

[[media/szz-11/media/image12.png]]

[[media/szz-11/media/image1.png]]

Midpoint algoritmus

Bresenham ale pro kružnici. Pracuje se s celými čísly, snadná implementace.

[[media/szz-11/media/image10.png]]

Polygon (vyplňování uzavřených oblastí)

Proces nalezení a označení (obarvení) všech vnitřních bodů dané oblasti. Vstupem je ohraničení oblasti, výstupem je rastrový popis vyplněné oblasti.

Druhy výplní: barvou (solid), šrafovou (hatch), texturou, gradientem [[media/szz-11/media/image19.png]]

[[media/szz-11/media/image16.png]]

Řádkové vyplňování (Scanline Fill)

Základní algoritmus pro vyplňování obecných mnohoúhelníků. Vstupem je **orientovaný seznam vrcholů**, výstupem vyrasterizovaný polygon (**mnohoúhelník**). Princip rasterizace je následující:

- Vyhledávají se **maximální** a **minimální** hodnoty souřadnic **X** a **Y** těchto bodů, čímž se **ohraničí oblast**, ve které se polygon nachází.
- Oblast se prochází **po řádcích** (na konci řádku se **resetuje** čítač hran).
- Počítají se průsečíky s hranami. Pokud je aktuální **počet** průsečíků **lichý**, řádek se **obarvuje**.

Problémy jsou s **lokálními extrémy** a **vodorovnými hranami**. Vodorovné hrany se **přeskakují**, problém lokálních extrémů lze řešit tak, že jsou všechny hrany **zkráceny** ve směru **osy Y** z obou stran o **1 pixel** (poté je nutné obrazec obtáhnout), nebo že je **započítán** v jednom bodě **průsečík obou hran**.

Inverzní řádkové vyplňování

[[media/szz-11/media/image4.png]]

Vstupem je seznam hran nebo orientovaný seznam vrcholů. Není nutné na každém řádku testovat hrany polygonu. Postup je následující.

- Vyhledávají se **maximální** a **minimální** hodnoty souřadnic **X** a **Y** těchto bodů, čímž se **ohraničí oblast**, ve které se polygon nachází.
- Pro každou hranu se získá souřadnice **Ymin** a **Ymax**.
- Pro každou hranu se hledají **průsečíky s jednotlivými řádky** (X-ové souřadnice), pixely **napravo** od průsečíku se invertují (0 černá, 1 bílá).
- Překreslení (obtáhnutí) obrysu.

Po provedení postupu pro všechny hrany bude vybarven **pouze vnitřek** polygonu. Problém může být v zasažení oblasti mimo polygon. Umožňuje pouze binární obraz.

[[media/szz-11/media/image14.png]]

Pinedův algoritmus (J. Pineda, 1988)

Pracuje pouze s **konvexními** mnohoúhelníky - trojúhelníky (ty jsou **vždy** konvexní). Umožňuje snadnou realizaci v HW. Vyplňovaná oblast je popsána seznamem hran a vyplňování probíhá:

- **Rozdělení** oblasti každou hranou na **poloroviny (kladnou a zápornou)**.
- **Vybarveny** jsou body (pixely) oblasti, které leží v **kladné polorovině** všech hran.

Semínkové vyplňování

[[media/szz-11/media/image7.png]]

[[media/szz-11/media/image15.png]]

Vyplňovaná oblast musí být definovaná **spojitou hranicí** z pixelů **požadované barvy**. Obravují se pouze pixely s barvou pozadí následovně:

- Semínko uvnitř oblasti **šíříme** na sousedy v okolí (**obarvování** sousedních pixelů).
- Obarvené pixely se **rekurzivně** stávají semínky (problém rekurze - může selhat, lze využít frontu a rekurzí odstranit).

používají se dva druhy okolí **4-okolí** a **8-okolí**, 8-okolí vyžaduje lépe definovanou hranici.

[[media/szz-11/media/image20.png]]

Křivky

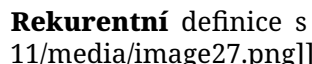
[[media/szz-11/media/image11.png]]

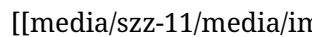
- **Interpolační** - Křivka prochází stanovenými (řídícími) body.
- **Aproximační** - Křivka nemusí procházet **řídícími** body - Beziérový křivky.

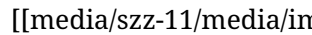
Beziérový křivky

- aproximační křivka (2D grafika, fonty)
- polynomiální křivka využívající Bernsteinových polynomů,
- křivka stupně **n** určena **n+1** body,

- prochází koncovými body.

Rekurentní definice s použitím **Bernsteinových** polynomů. Používá se pro fonty. 

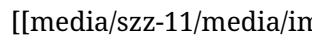


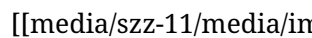


Algoritmus de Casteljau

Rekurzivní algoritmus pro vykreslování **Beziérových křivek** (plyne z rekurentní definice Bernsteinových polynomů). Postup:

- Zvolí se dostatečně jemný krok, se kterým se mění hodnota **t** (t náleží $(0,1)$).
- Úseky polynomů se dělí v poměru **t** a **1-t**, v místě dělení vznikne **nový bod** pro další dělení. **Snižuje** se tak s každým krokem **stupeň polynomu**, až na konci zbyde pouze jeden **bod**.
- Vzniklé body se **spojí úsečkami** (tvar křivky závisí na použitém kroku).

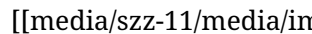




Bézierovy kubiky

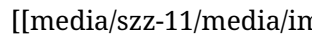
Beziérová křivka **3. stupně** popsána Bernsteinovými polynomem 3. stupně. Segment je popsán **4 řídicími body**.

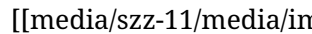
- **Divide and Conquer - De Casteljau** a Bézierovy kubiky. Rekurzivní dělení na **2 podkřivky**, dostatečně rovná křivka se dál nedělí a vykreslí se.



Navazování segmentů Beziérových kubik

Vyžaduje **totožné koncové body** a **stejně tečné vektory** v navazujícím bodě. **Koncový bod je středem úsečky mezi předposledním bodem první křivky a druhým bodem druhé křivky.**





Odkazy:

- Modelování křivek
- Animace Bézierova křivka
- De Casteljau animace (video)
- De Casteljau's Algorithm and Bézier Curves

Zdroje

- SZZ okruh 11 — studijní materiály FIT BUT (szz-11.docx). Další obrázky: media/szz-11/.

Navigace: [\[\[index| • Přehled okruhů\]\]](#) · [\[\[okruhy/10-fpga-vhdl|10. Technologie FPGA\]\]](#) · další: [\[\[okruhy/12-3d-transformace-pipeline|12. Transformace 3D modelů\]\]](#) [\[\[okruhy/12-3d-transformace-pipeline|12. Transformace 3D modelů\]\]](#)